Component-Level Inverse Design of Transmon Qubits Using Neural Networks

Olivia Seidel[1,2*], Firas Abouzahr[3], Abhishek Chakraborty[4,5], Sadman Ahmed Shanto[6,7], Saikat Das[6,7], Sara Sussman[1], Daniel Baxter[1,3], Nicola Pancotti[10], Haoyu Yang[10], Brucek Khailany[10], Enectali Figueroa-Feliciano[1,3], Eli M Levenson-Falk[6,7,8,9], Taylor L. Patti[10]

[1*]Fermi National Accelerator Laboratory, PO Box 500, Batavia, 60510, IL, USA.
[2]Department of Physics, University of Texas at Arlington, 502 Yates Street, Arlington, 76019, TX, USA.
[3]Department of Physics and Astronomy, Northwestern University, 2145 Sheridan Road, Evanston, 60208, IL, USA.
[4]Institute for Quantum Studies, Chapman University, One University Drive, Orange, 92866, CA, USA.
[5]Department of Physics and Astronomy, University of Rochester, 500 Wilson Boulevard, Rochester, 14627, NY, USA.
[6]Center for Quantum Information Science and Technology, University of Southern California, 925 Bloom Walk, Los Angeles, 90089, CA, USA.
[7]Department of Physics & Astronomy, University of Southern California, 825 Bloom Walk, Los Angeles, 90089, CA, USA.
[8]Ming Hsieh Department of Electrical & Computer Engineering, University of Southern California, 3740 McClintock Avenue, Los Angeles, 90089, CA, USA.
[9]Quantum Elements, Inc, Thousand Oaks, CA, USA.
[10]Research, NVIDIA Corporation, 2788 San Tomas Expressway, Santa Clara, 95051, CA, USA.

*Corresponding author(s). E-mail(s): olivias@fnal.gov;
Contributing authors: firasabouzahr2030@u.northwestern.edu; achakraborty@chapman.edu; shanto@usc.edu; saikatda@usc.edu;

sarafs@fnal.gov; dbaxter9@fnal.gov; npancotti@nvidia.com;
haoyuy@nvidia.com; bkhailany@nvidia.com; enectali@northwestern.edu;
elevenso@usc.edu; tpatti@nvidia.com;

Abstract

Designing a superconducting qubit to realize a specific Hamiltonian typically requires iterating through a forward loop in which a geometry is chosen, simulated, converted into circuit quantities, and then adjusted. We study the inverse version of this task for a single transmon layout. The aim of this work is to predict component-level Quantum Metal parameters directly from target Hamiltonian parameters. We focus on a planar transmon qubit with a cross-shaped capacitor island generated with the Quantum Metal workflow, and train a multilayer perceptron (MLP) inverse model using data from the SQuADDS database. The inverse model is paired with a learned surrogate MLP model for the Ansys finite-element extraction step, which, once trained, allows candidate designs be checked without running new Ansys simulations. During training, the inverse MLP takes target values for qubit frequency and anharmonicity as input, predicts Quantum Metal geometry parameters as output, and contains a validation loss facilitated through a frozen forward surrogate. In this manner, the loss is evaluated in Hamiltonian space rather than in Quantum Metal geometry space. On the test set, the combined pipeline reaches mean percent errors of 0.94% for qubit frequency and 2.04% for anharmonicity. These errors are comparable to the fabrication and simulation-to-measurement uncertainty expected for academic-process transmon devices of this type. Our fully AI-mediated workflow evaluates a solution in just $\sim$56 ms on CPU, while a single Ansys Q3D capacitance extraction takes about $\sim$2 min on the same workstation, providing a speedup of more than 2,100$\times$. When many samples are evaluated simultaneously, the overhead call to tensorflow is minimized, reducing the full runtime to 0.235ms per sample. Our results indicate that component-level inverse design is useful even for small datasets.

Keywords: inverse design, superconducting qubits, transmon, Quantum Metal, neural network surrogate, SQuADDS

1 Introduction

The superconducting-qubit design space remains largely underexplored because the design process is still mostly a forward problem. That is, the designer sketches a layout in a tool such as Quantum Metal (formerly known as Qiskit Metal) [1], renders it for finite-element electromagnetic extraction [2], extracts capacitance and mode properties, computes the corresponding Hamiltonian parameters from those quantities [3], and then intuitively adjusts the geometry based on the result [4, 5]. Such a loop is not only heuristic and non-systematic, it is costly in both compute and human time. Moreover, the reliance on human intuition and expensive simulations is particularly problematic due to the fact that small geometric changes often have large and even counter-intuitive effects on the effective circuit parameters. Although the transmon is a well-characterized qubit modality

[6], reaching a target Hamiltonian from a manufacturable transmon layout still requires both researcher experience and repeated simulations.

The more direct route is to learn an inverse map from desired Hamiltonian parameters to Quantum Metal geometry parameters. Closely related strategies are used in nanophotonics, where AI models have been used both as fast surrogates for full-wave solvers and as inverse predictors for device geometries [7–9]. One complication is that the inverse problem is not unique. Many hardware geometries can produce nearly the same Hamiltonian response. A model trained only to reproduce one reference geometry for each target can therefore average over incompatible solutions and return a design that is not physically useful [8]. Recent works have addressed this by connecting an inverse network to a pretrained forward network and training the inverse model through the predicted response rather than through geometry matching alone [8]. We use the same basic idea here, but apply it to Quantum Metal transmon design.

Similar ideas are starting to appear in superconducting-circuit workflows. Recent work has used machine-learning surrogates to accelerate qubit readout-fidelity simulations and to support readout-system inverse design [10, 11]. However, our application is distinct: we target transmon qubit Hamiltonians themselves and do so at the component level, predicting Quantum Metal parameters that can be rendered and checked through the SQuADDS open-source qubit design database [4].

This paper presents a Hamiltonian-targeted inverse-design workflow for one Quantum Metal qubit layout. Target Hamiltonian parameters are passed to a trained MLP, which predicts Quantum Metal geometry parameters. The predicted design can then be checked either with the learned Ansys surrogate or with the original Ansys-based extraction path. While the main text focuses on the transmon cross, the component with the largest SQuADDS-derived dataset in this study, the appendices also include results from the other SQuADDS datasets.

In this manuscript, we make the following contributions:

- A full inverse-design pipeline that predicts transmon-cross Quantum Metal parameters, reconstructs capacitance and mode derived circuit quantities, and converts them back to to Hamiltonian targets using lightweight transmon relations.
- A direct inverse MLP that predicts the transmon-cross geometry parameters from a target qubit frequency and anharmonicity, which serves as the main demonstration for this paper.
- A learned Ansys surrogate that replaces the Q3D capacitance-extraction step during neural validation and enables fast evaluation.
- Hamiltonian validation of the combined inverse+surrogate workflow on SQuADDs transmon-cross samples, with the errors reported on the qubit frequency and anharmonicity, showing how closely the predicted parameters recover the target quantities. The main text reports Hamiltonian-level results, while the appendices report capacitance-level results as well as additional inverse design studies performed on SQuADDS resonator and coupling-capacitor datasets.
- A small-data scaling analysis, together with a discussion of inverse non-uniqueness in cases where distinct geometries give nearly identical Hamiltonian parameters.

- Additional inverse-design MLP models and studies for the coupling capacitor and resonator components of a chip, included in the appendices, to complete a fully AI-driven qubit-resonator design.

Section 2 defines the target quantities, the physics-based mappings, and the inverse-design workflow. Section 3 describes data generation and preprocessing. Section 4 gives the model and training procedure. Section 5 reports the transmon-cross results and small-data analysis, followed by discussion, limitations, and conclusions. The appendices collect the inverse-only baseline and the NCap and resonator studies.

2 Theory and workflow

Figure 2 summarizes the full workflow. The user-desired "target" Hamiltonian parameters are passed to the inverse MLP, the MLP returns component-level Quantum Metal parameters, and the resulting layout is assembled and validated through forward extraction or through the learned surrogate.

The network inputs are the transmon Hamiltonian parameters, specifically the qubit frequency ω_q and the anharmonicity α. For the resonator in Appendix E the inputs are the resonator frequency ω_r and the linewidth κ. The NCap coupling capacitor uses the six capacitance-matrix entries directly as inputs, and is covered in Appendix D.

2.1 Deriving Hamiltonian targets from the capacitance matrix

The map from Hamiltonian targets back to Quantum Metal parameters is one-to-many. Different capacitance matrices, and therefore different layouts, can give nearly the same Hamiltonian values. If the inverse model is trained only with a direct geometry-regression loss, several valid designs may compete for the same target, and the network can average them into a poor design. This failure mode is well known in nanophotonic inverse design [7–9]. To avoid this issue, we construct our inverse model by first training a forward surrogate that takes Quantum Metal parameters as input and outputs Hamiltonian parameters. We can then freeze the parameters of this forward surrogate, insert it inverse model, and then evaluate the loss in Hamiltonian space.

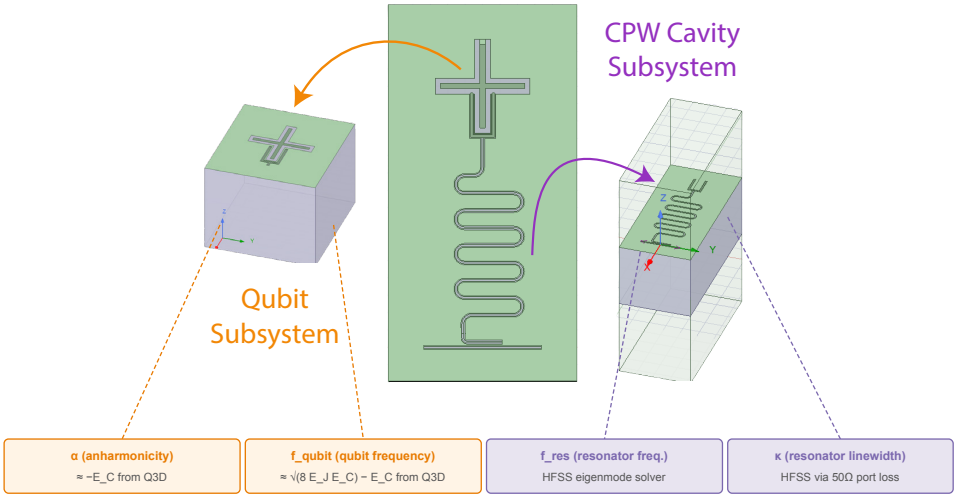
The effective Hamiltonian of the coupled transmon-resonator system defines our target superconducting qubit quantities. Let $\hat{n}_r = \hat{a}^\dagger\hat{a}$ and $\hat{n}_q = \hat{b}^\dagger\hat{b}$, such that

$$\begin{aligned}\hat{H} = \hbar\omega_r\left(\hat{n}_r + \tfrac{1}{2}\right) + \hbar\omega_q\hat{n}_q \\ + \frac{\alpha}{2}\hat{n}_q\left(\hat{n}_q - 1\right) \\ + \hbar g\left(\hat{a} - \hat{a}^\dagger\right)\left(\hat{b} - \hat{b}^\dagger\right),\end{aligned} \tag{1}$$

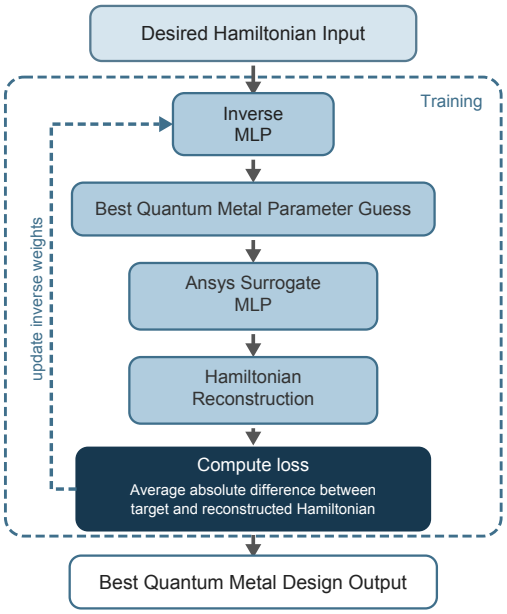
where ω_r is the resonator frequency, ω_q is the qubit transition frequency, α is the qubit anharmonicity, and g is the qubit-resonator coupling. In the transmon limit, the relations

$$\begin{aligned}\omega_q \approx \sqrt{8E_JE_C} - E_C, \\ \alpha \approx -E_C.\end{aligned} \tag{2}$$

give useful analytic intuition for the dependence on. In the pipeline used for the reported results, however, ω_q and α are computed with the scQubits lumped-oscillator-model (LOM) workflow by numerical diagonalization of the Hamiltonian, with E_C derived from the extracted capacitance matrix and E_J held fixed. The charging energy is set by the



(a) The single qubit transmon and resonator system studied in this work, adapted from the SQuADDS framework [4].



(b) Forward pass with the learned Ansys surrogate replacing Ansys in the loop.

Fig. 1: Overview of the inverse design setup. (a) The single qubit transmon and resonator layout studied in this work. (b) The fully neural forward pass, with a trained Ansys surrogate MLP replacing the EM solver. The Ansys in the loop testing pipeline is shown in Appendix A.

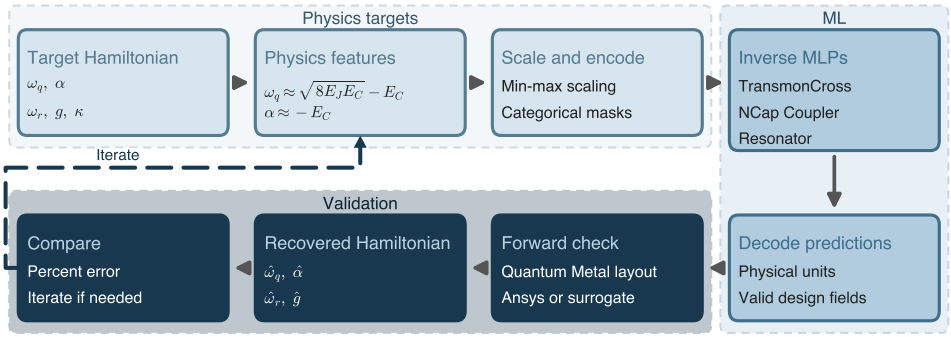


Fig. 2: End-to-end inverse-design workflow, grouped by role. Physics-based feature construction is shown in orange, component-specific ML surrogates in green, and forward-simulation validation in purple. The right column shows the data object flowing into each stage.

total qubit capacitance C_Σ through $E_C = e^2/(2C_\Sigma)$, and the Josephson energy E_J is set by the junction inductance L_J. The resonator frequency ω_r and the linewidth κ come from the HFSS eigenmode solver.

2.2 Outputs and Quantum Metal parameters

The transmon cross inverse model predicts three Quantum Metal design parameters that together set the qubit geometry: the coupling claw length and ground spacing control the coupling strength between the qubit and its readout resonator, while the cross length controls the overall size of the transmon cross, which is the main parameter dictating the qubit capacitance and therefore E_C. Value ranges and units are given in Table 1.

2.3 Targets and Hamiltonian parameters

For the transmon cross, the Hamiltonian targets are the qubit frequency ω_q and anharmonicity α. These quantities are the most directly tied to the cross geometry through E_C, while E_J is fixed by the junction inductance of 10 nH, making them the cleanest targets for isolating the qubit component. Resonator frequency and coupling-related quantities are handled separately in the resonator study in Appendix E.

3 Data generation and preprocessing

3.1 Component datasets and sampling strategy

We use the SQuADDS database [4] as the the component dataset source, and as the reference workflow for forward validation. Starting from the full SQuADDS parameter set, we keep only the fields that vary within the component sweep as they are used in the inverse-design task studied here. Parameters that are not predicted by the model are held at their SQuADDS default values during forward validation.

Table 2 summarizes the transmon cross dataset used in the main text, along with the NCap and resonator datasets covered in the appendices. After filtering, all

Table 1: Quantum Metal output parameters (**y**) predicted by the transmon cross inverse model.

Quantum Metal Parameter	Value range	Units
Qubit model, TransmonCross (cap_matrix)		
`design_options.connection_pads.readout.claw_length`	70 to 400	μm
`design_options.connection_pads.readout.ground_spacing`	4 to 10	μm
`design_options.cross_length`	90 to 420	μm

three datasets use a 70/15/15 train/validation/test split. The models trained on the significantly smaller NCap and resonator datasets are reported in Appendix D and Appendix E, respectively.

The parameter ranges covered in the SQuADDS datasets are given in Table 1. Parameters that are constant across the sweep are removed before training, leaving only fields that vary from sample to sample. We create the train, validation, and test split once, keeping these data subsets consistent throughout the text. Figure 3 shows that the fixed split preserves the overall parameter coverage across the three Quantum Metal outputs.

Figure 3 shows the SQuADDS parameter distributions used for the transmon cross, separated by train, validation, and test split. The claw length is biased toward smaller values, ground spacing is sampled only on a coarse grid, and cross length is more evenly distributed. The similar split-wise distributions indicate that the fixed 70/15/15 split preserves the overall parameter coverage used for training and evaluation. The discrete ground-spacing values make that output easier to learn inside the sampled grid, but they also limit what can be inferred about values between grid points.

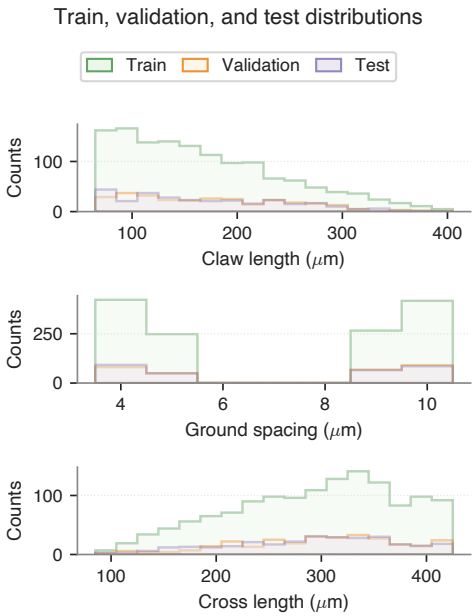


Fig. 3: Distributions of the three Quantum Metal output parameters used for the transmon-cross inverse model, separated by train, validation, and test split. The splits follow the fixed 70/15/15 partition used throughout training and evaluation.

3.2 Simulation pipeline

Each training sample starts from a Quantum Metal design generated by SQuADDS [4] at a given set of design parameters. The corresponding design is rendered into Ansys, where Q3D provides the capacitance matrix for the qubit and HFSS provides the eigenmode frequencies for the cavity. The total qubit capacitance C_Σ is computed from the Q3D capacitance matrix, and the charging energy

Table 2: Component datasets used in this work. Each dataset comes from SQuADDS after filtering parameters that are held fixed across the sweep, and is split 70/15/15 into training, validation, and test sets. Train / val / test counts come directly from the notebooks used to train each model.

Component	Total	Train	Val	Test	Forward extraction	Location
TransmonCross (qubit)	1934	1353	290	291	Ansys Q3D (capacitance matrix)	Main text
NCap (coupling cap.)	430	301	64	65	Ansys Q3D (capacitance matrix)	App. D
Cavity / resonator	810	567	121	122	Ansys HFSS (eigenmode)	App. E

is obtained from $E_C = e^2/(2C_\Sigma)$. This E_C, together with a fixed E_J, is passed to scQubits [3], which then performs a lumped-oscillator-model (LOM) analysis by numerically diagonalizing the transmon Hamiltonian to return ω_q and α. All other geometric and material choices are kept fixed across the sweep, including the chip substrate (silicon, with cryogenic permittivity), the ground-plane padding, and the junction inductance. These are left at their SQuADDS defaults during forward validation, so that only the parameters predicted by the network vary.

Each sample therefore contains both the Quantum Metal parameters used as model outputs and the Hamiltonian quantities used as model inputs. Q3D convergence is monitored through the relative error between adaptive passes. Samples that fail to reach the convergence threshold within the pass budget are removed before training.

3.3 Scaling and normalization

We apply min-max normalization to both inputs and outputs. The scalers are calculated from the training set only and then applied to the validation and test sets, so that no information leaks across splits. Since the network is trained in scaled space, we save the fitted scalers and reapply the inverse transform to predictions at evaluation time, bringing outputs back into physical units for use in downstream tasks and model evaluation.

4 Models and training procedure

4.1 Model selection

We use multi-layer perceptrons (MLPs) for both the Ansys surrogate and inverse models because the inputs and outputs of both modeling tasks are low-dimensional numerical parameters from modestly sized datasets. In this regime, larger architectures are often not advantageous, as they are prone to overfitting, particularly in sparse parameter space regions. We were likewise encouraged to explore MLPs because fully connected networks of comparable scale are common architecture choices in the nanophotonic inverse-design literature [7–9].

4.2 Model specifications

The inverse model MLP has two hidden layers of width 16, LeakyReLU activations, and no dropout, yielding just 387 trainable parameters in total. The input dimension is 2, matching ω_q and α, and the output

dimension is 3, matching the three Quantum Metal parameters in Table 1. The Ansys forward surrogate is a larger MLP with 736 hidden units and 4,418 parameters, trained separately on the same SQuADDS-derived dataset to map Quantum Metal parameters forward to the Hamiltonian targets ω_q and α. When the inverse desigm MLP is trained in the combined pipeline shown in Fig. 4, the forward surrogate is frozen and only the inverse-MLP weights are updated. This training structure is analogous to the one Liu et al. used for nanophotonic inverse design [8], adapted here to Quantum Metal design variables.

4.3 Loss functions and optimization

We tested both MSE and MAE objectives. MAE gave slightly more stable training and better held-out performance, likely because a small number of geometries lie in sparsely covered regions of SQuADDS and behave like outliers under squared error. All models are trained with Adam at a learning rate around 10^{-3}, batch size 32, and early stopping conditioned on validation loss increase. We use light L2 regularization to keep the weights small, and the Keras Tuner is used to optimize the model hyperparameters, as described in Section 4.4.

The sweep in Fig. 5 has its best mean validation loss at depth 1 and width 32. Increasing either depth or width does not improve the result. If capacity were the main bottleneck, larger models should reduce validation loss. Instead, the loss flattens and then worsens, consistent with the small-data behavior discussed in Section 5.3.

4.4 Hyperparameter tuning (Keras Tuner)

We used Keras Tuner to optimize over network depth (1 to 5 hidden layers), width (16 to 512 neurons per layer), learning rate (10^{-4} to 10^{-2}), L2 regularization coefficients, dropout rate, a penalty weight, batch normalization, and total trainable parameter count. The best validation loss from our ablation studies was 0.0008 and came from a network with two hidden layers of 16 neurons each. This is the configuration we use for our reported results in the subsequent sections. Figure 6 summarizes the search.

4.5 Evaluation metrics

We report root mean squared percentage error (RMSPE), along with the mean, median, and interquartile range of the per-sample percent error between target and achieved quantities. This metric is chosen due to the large difference in scales between the various parameters, e.g., qubit frequency in the few GHz range versus an anharmonicity in the few hundred MHz range. Due to the non-Gaussian nature of the resulting error distributions, we illustrate our results with box plot of the per-sample percent error with the medians and means given in the legend.

4.6 Runtime and speedup

As electromagnetic extraction is the computational bottleneck insuperconducting qubit design loops, simulation and design acceleration is a primary motivation of our AI-based approach. To quantify the speedup obtained from a surrogate model, we benchmarked each step of both the conventional pipeline and the fully neural pipeline on the same hardware (an Intel

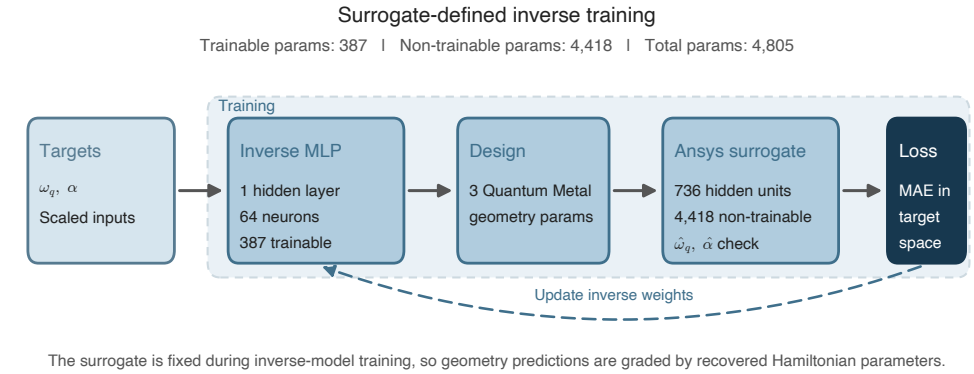


Fig. 4: Illustrative model architecture.

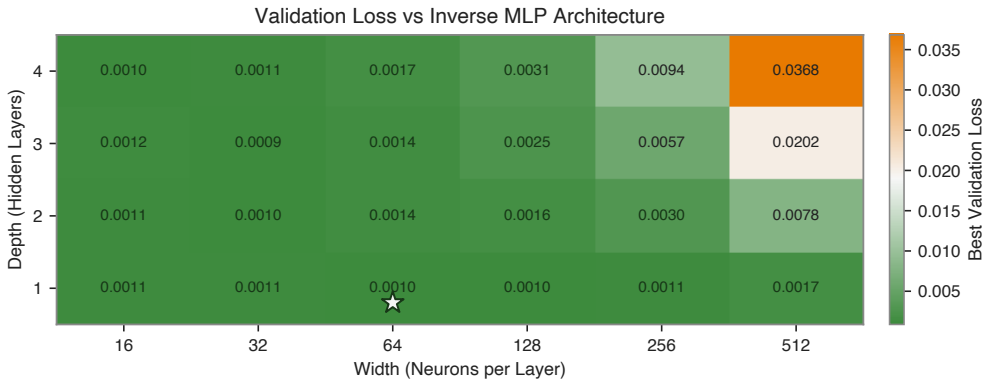


Fig. 5: Hyperparameter sweep results for the transmon cross model.

Xeon W-2245, 8-core Windows workstation running Python 3.11.14 with TensorFlow 2.20, CPU-only inference).

The runtime of both methods on the aforementioned hardware is detailed in Table 3. In particular, a single Ansys Q3D extraction on the transmon cross takes approximately 2 minutes, depending on the geometry and the final mesh size. Conversely, a single call to the combined inverse plus surrogate pipeline takes about 56 ms, meaning that generating and validating one candidate design through our learned forward surrogate approximately roughly three orders of magnitude faster than completing the same task on Ansys. Furthermore, as the single-call number is dominated by TensorFlow per-call dispatch overhead rather than by the actual network math, when the same pipeline is run on batches of samples (as would be the case for virtually any model training protocol), the per-sample cost drops to about 0.235 ms. The added acceleration of batched evaluation is even more stark for the forward surrogate component of the inverse network, droping to about

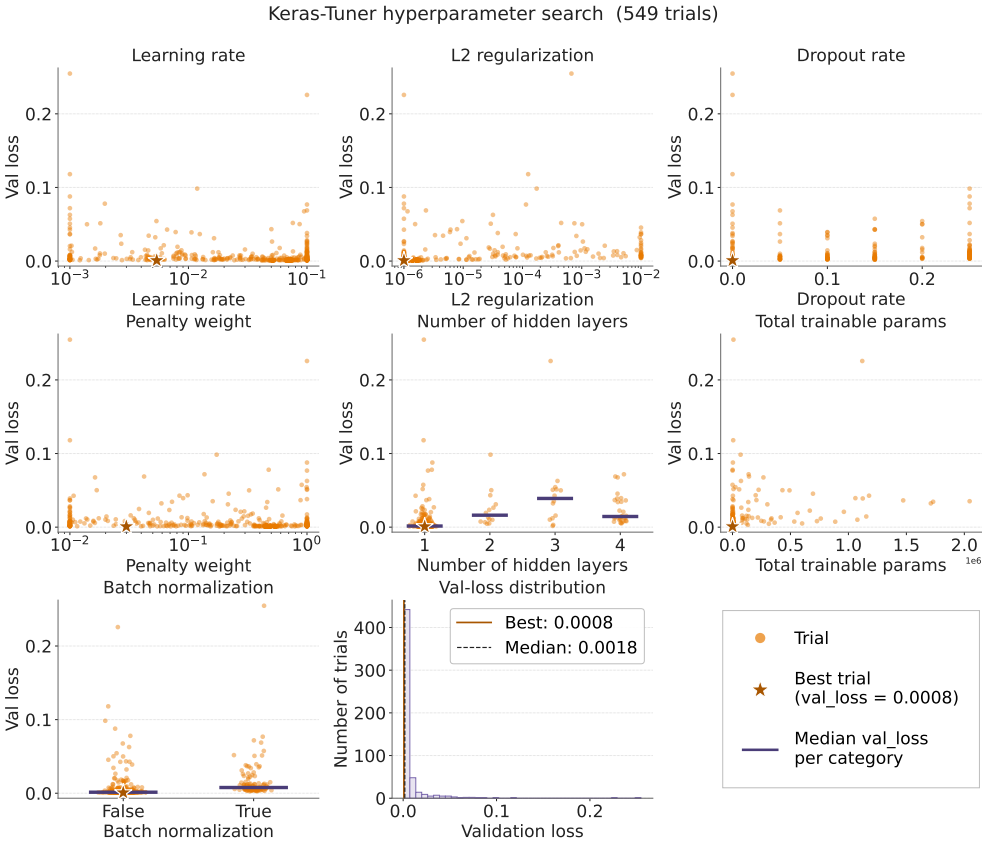


Fig. 6: Illustrative hyperparameter search metrics for the transmon cross model.

24.437 μs per sample for batches of size 50,000.

This speedup further compounds when the design loop is iterative. For example, the nearest-neighbor stress test in Section 5.3 evaluates 50,000 candidate designs through the learned surrogate before selecting 90 high-scoring candidates for Ansys validation. This entire in-depth search takes only seconds using our Ansys surrogate model. Conversely, evaluating all 50,000 candidates directly through Ansys Q3D would take about 1,667 CPU-hours.

We report inference on CPU rather than GPU intentionally, because Ansys Q3D is a CPU workload for most users. On a GPU the neural pipeline would get even faster, but the Ansys baseline would not move, so the comparative speedup would only grow.

5 Results

5.1 Reference uncertainty: simulation-to-fabrication and measurement mismatch

The model errors should be interpreted relative to the error floor of the simulation and fabrication stack. That is, AI models

Table 3: Per-design runtime for each stage of the conventional and neural inverse-design pipelines, measured on an Intel Xeon W-2245, 8-core Windows workstation (Python 3.11.14 with TensorFlow 2.20, CPU-only). The Ansys Q3D timing is the mean time per sample during SQuADDS training-data generation on the same hardware. Neural timings are the mean of 50 calls per stage, after 10 warm-up calls. Batched per-sample figures use the 291-sample held-out test set for the inverse and combined pipelines, and 50,000 uniform, in-distribution random samples for the forward surrogate alone.

Stage	**Single call**	**Per-sample (batched)**	**Notes**
Conventional, Ansys in the loop			
Ansys Q3D capacitance extraction	∼2 min	∼2 min	Default SQuADDS mesh and pass budget
Hamiltonian mapping from capacitances	< 1 ms	< 1 ms	Analytic
Neural pipeline (this work)			
Inverse MLP inference	56.34 ms	0.223 ms	2 in, 3 out, 387 params
Forward surrogate MLP inference	55.28 ms	0.0244 ms	3 in, 2 out, 4,418 params
Combined inverse + surrogate (end-to-end)	56.12 ms	0.235 ms	Fully neural closed loop
Hybrid inverse + single Ansys validation	∼2 min	Not Applicable	Final model ready for use

can only be as acurate as the data used to train them. The origins of such inherent errors are numerous. For example, simulated capacitances and mode frequencies do not match fabricated devices exactly. Likewise, meshing choices, truncation of the full circuit to an effective Hamiltonian description, finite element discretization, and differences in extraction conditions further accumulate into a few percent of uncertainty before the inverse model is even in the picture. The inherent uncertainty of superconducting qubit design further augments when we move from theoretical simulations and into experimental nonidealities. Such experimental nonidealities are many, and include phenomena such as the lithographic variability and junction area uncertainty. As a result, measured Hamiltonian parameters typically differ from their simulated counterparts by at least a few percent [4].

One source of fabrication error relevant to this work is the precision limitation of the lithography tool used to write the qubit capacitor and surrounding ground plane. For instance, a direct-write lithography tool found in academic cleanrooms such as the Heidelberg DWL has a precision of order $1\,\mu$m. For a transmon cross with characteristic dimension $\ell \sim 250\,\mu$m, this propagates to a fractional capacitance uncertainty of $\delta C/C \sim 2\,\delta\ell/\ell$, of order 1%, since the capacitance scales roughly linearly with the size of the cross [5]. Through $E_C = e^2/(2C_\Sigma)$ this translates directly into a $\sim 1\%$ shift in $\alpha \approx -E_C$ and a $\sim 0.5\%$ shift in $\omega_q \approx$

$\sqrt{8E_JE_C} - E_C$, consistent with the few-percent simulation-to-measurement floor reported by SQuADDS [4]. An additional, typically larger bias is introduced by the subsequent etch step, which systematically removes material beyond the written resist edge and shifts each feature by a process-dependent amount. The magnitude of this contribution is process-specific and often treated as a confidential foundry parameter. Its effect can still be seen experimentally. Even for foundry-quality fabrication, uncompensated etch bias alone produces a $\sim 0.3\%$ simulation-to-measurement discrepancy in resonator frequency [5], comparable in magnitude to the lithography contribution derived above. Combining the two, the total fabrication-limited uncertainty on α is roughly 1 to 2% and on ω_q roughly 0.5 to 1%. The closed-loop errors we report below should be read against this combined fabrication-limited floor rather than against a true zero.

5.2 Inverse + surrogate transmon cross, Hamiltonian-level performance

We evaluate the closed-loop inverse-plus-surrogate pipeline on the transmon cross by comparing target and reconstructed Hamiltonian parameters, namely qubit frequency ω_q and anharmonicity α. The capacitance-level comparison, using exactly the same samples but looking at extracted capacitance matrix elements instead of derived Hamiltonian values, is in Appendix C.

Across the held-out test set, the mean percent error is 0.94% for ω_q and 2.04% for α, with medians of 0.87% and 1.90% respectively. For a 4 to 6 GHz qubit, this corresponds to frequency errors of only a few tens of MHz, while the anharmonicity error is on the order of a few MHz for a typical 150 to 200 MHz target. These errors reproduce the factor-of-two asymmetry expected from the fabrication analysis in Section 5.1. Anharmonicity inherits the full fractional capacitance uncertainty directly, while qubit frequency inherits roughly half, since the dominant $\sqrt{8E_JE_C}$ contribution to ω_q suppresses the fractional sensitivity to E_C by a factor of two. Their absolute magnitudes also sit within the combined lithography-plus-etch floor

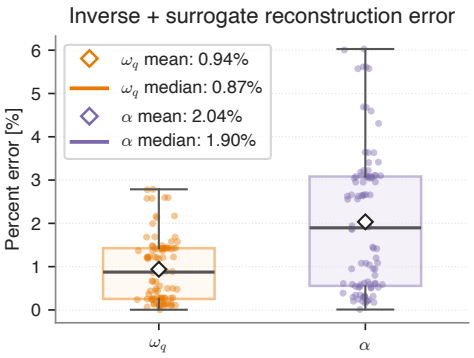


Fig. 7: Closed-loop Hamiltonian-level performance of the combined inverse and surrogate pipeline on the transmon cross. Box plots show the distribution of per-sample percent error between target and achieved qubit frequency ω_q and anharmonicity α on the held-out test set, with individual samples overlaid as jittered points. Medians (heavy line) and means (diamond) are reported in the legend. Note that the inverse model's predicted geometry can differ substantially from the SQuADDS reference geometry even when the reconstructed Hamiltonian agrees closely with the target. This behavior follows from the one-to-many inverse map rather than from a failure of the model. An explicit geometry comparison for one such sample is shown in Fig. 12.

for typical academic fabrication processes [4, 5] (roughly 1 to 2% for α and 0.5 to 1% for ω_q, Section 5.1), with 2.04% and 0.94% landing at the upper end of each range. Within the SQuADDS-sampled region, further reductions in test-set model error would likely be difficult to observe experimentally unless fabrication variability and simulation-to-measurement mismatch are also reduced.

5.3 Small-data scaling and stress test

To test how strongly the inverse-plus-surrogate performance depends on local training set coverage, we generated random Quantum Metal parameter sets inside the physically valid region of the transmon-cross design space. The test samples candidate geometries directly within the training-domain bounding box, rejecting designs that are outside the space spanned by the training data. This makes the test an interpolation stress test rather than an extrapolation test. Each candidate point is scaled to the normalized training coordinates and evaluated by the surrogate model. We then compute the Euclidean distance from each candidate to its nearest training sample in scaled Quantum Metal parameter space. This nearest-neighbor distance serves as a simple measure of how locally supported the candidate point is by the SQuADDS training distribution. Small distances correspond to densely sampled regions in the training data space, while large distances correspond to sparse regions that may not be predicted as well with the surrogate model.

From 50,000 valid uniformly sampled candidates, we divide the nearest-neighbor distance range into 10 equal-width bins and select 9 candidates from each bin, giving 90 candidates for Ansys validation. We then compare the surrogate prediction to the Ansys simulated values, to determine how well the model performed. Fig. 8 summarizes this methodology. Fig. 10 then shows the selected validation points in Quantum Metal parameter space, colored by their scaled nearest-neighbor distance. Figure 10 compares surrogate-predicted Hamiltonian quantities against Ansys-validated quantities as a function of distance from the nearest training sample.

The nearest-neighbor stress test behaves in the way we would expect if local training-set coverage is one of the main limits on the surrogate. As the scaled distance from the nearest SQuADDS training sample increases, the Ansys-validated surrogate error also increases, with the clearest effect appearing in the anharmonicity. The qubit-frequency error stays fairly stable across most of the distance range, while the anharmonicity error grows more noticeably for candidates that sit in the sparsest parts of the sampled design space.

The fact that the Hamiltonian errors grow more slowly than the nearest-neighbor distance is also a useful check on what the surrogate has learned. If the model were mostly memorizing nearby training samples, or if a nearest-neighbor lookup were doing nearly the same job, the error would be expected to track the geometric distance more closely. Instead, the surrogate keeps the ω_q error around a few percent over much of the tested range, while the α error rises mainly in the least supported regions. This suggests that the MLP has learned a physically meaningful approximation of the relationship between geometry and Hamiltonian parameters, rather than simply copying or locally interpolating between individual SQuADDS entries. The increase at large

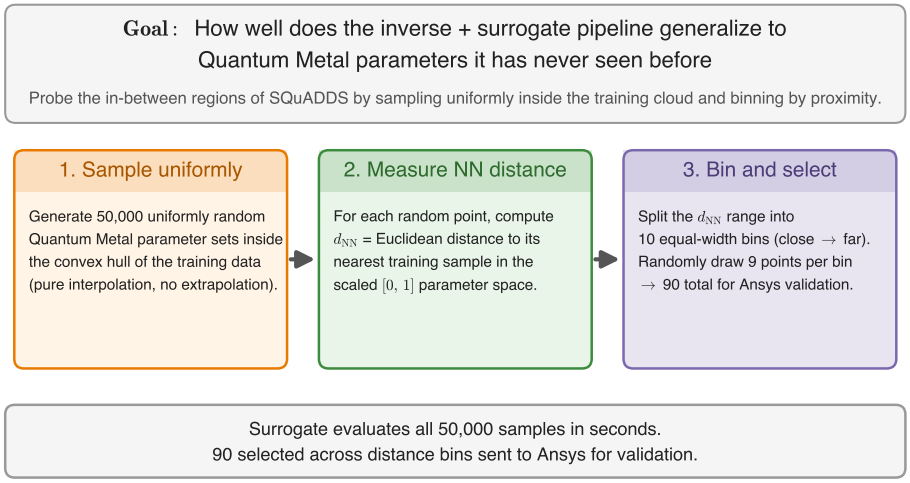


Fig. 8: Nearest-neighbor stress-test methodology. Uniformly random Quantum Metal parameter sets are generated inside the transmon-cross training data space, constrained to remain inside of the interpolated SQuADDS design domain. The learned surrogate evaluates 50,000 valid candidates, and each candidate is assigned a scaled Euclidean distance to its nearest training sample. The candidates are then split into 10 equal-width nearest-neighbor-distance bins, and 9 candidates are selected from each bin for Ansys validation, giving 90 validation designs in total.

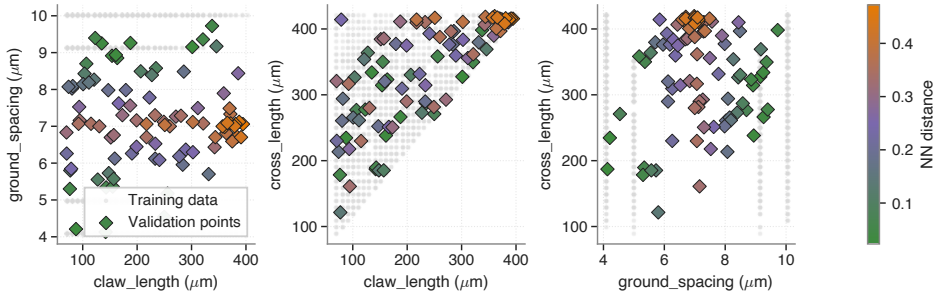


Fig. 9: Spatial visualization of the nearest-neighbor stress test in Quantum Metal parameter space. The grey points show the SQuADDS training distribution, and colored points show the 90 uniformly sampled validation candidates selected across 10 scaled nearest-neighbor-distance bins. The distance is computed in normalized Quantum Metal parameter space, so each predicted geometry parameter contributes equally. Candidates at larger nearest-neighbor distance remain inside the space spanned by the training data, but lie in more sparsely sampled regions where surrogate reliability is expected to degrade first.

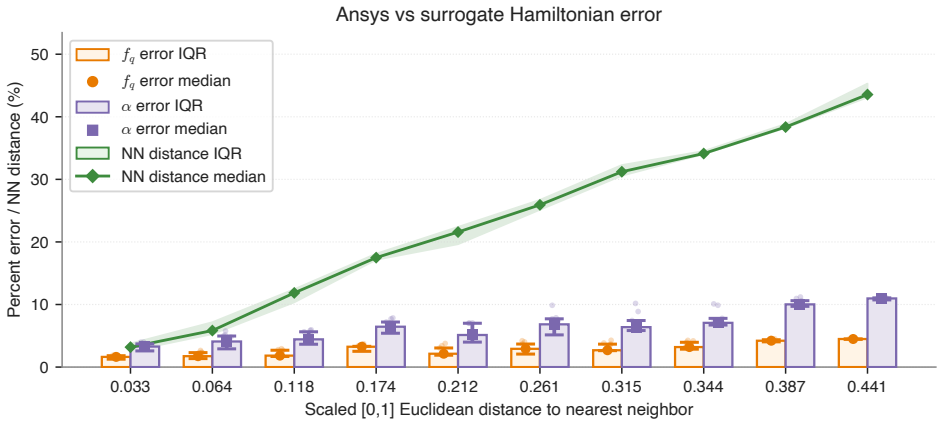


Fig. 10: Ansys validation of the surrogate stress test as a function of scaled nearest-neighbor distance in Quantum Metal parameter space. Orange and purple bars show Hamiltonian parameter agreement between the surrogate prediction and Ansys validated result for ω_q and α, with individual validation samples overlaid. The green curve shows the corresponding geometry similarity to the nearest training sample. The trend tests whether surrogate accuracy degrades as candidate designs move into more sparsely sampled, but still interpolated, regions of the SQuADDS design space.

nearest-neighbor distance is still important, though, and it reinforces that the surrogate is best used as a fast screening tool before final Ansys validation, rather than a complete replacement for it.

As a complement to the stress test, Fig. 11 shows what happens when the transmon-cross inverse model is trained on progressively larger fractions of the full training set, from 5% to 100%. The train, validation, and test losses fall quickly up to about 20% of the training set, or roughly 250 samples, and then change very little after that. This suggests that the model has already learned most of what it can from the current SQuADDS sampling pattern. In other words, adding more samples drawn in the same way would probably not lead to a large improvement.

The more useful takeaway is that the remaining error is tied to where the samples are located in design space, not just how many samples there are overall. This matches the nearest-neighbor stress test, where the surrogate error grows in sparsely sampled regions. Future simulations should therefore be chosen more deliberately, for example by targeting regions with poor local coverage, high surrogate uncertainty, or larger validation error. A better route is targeted data generation in regions where the surrogate is least supported or least reliable.

5.4 End-to-end examples

We also check individual designs by passing a single target specification through the inverse-plus-surrogate pipeline, assembling the resulting transmon-cross layout, and validating it with forward simulation. As expected from the one-to-many structure of the inverse problem, the predicted Quantum Metal geometry can differ noticeably from the SQuADDS reference geometry even when the reconstructed Hamiltonian lands very close to the target.

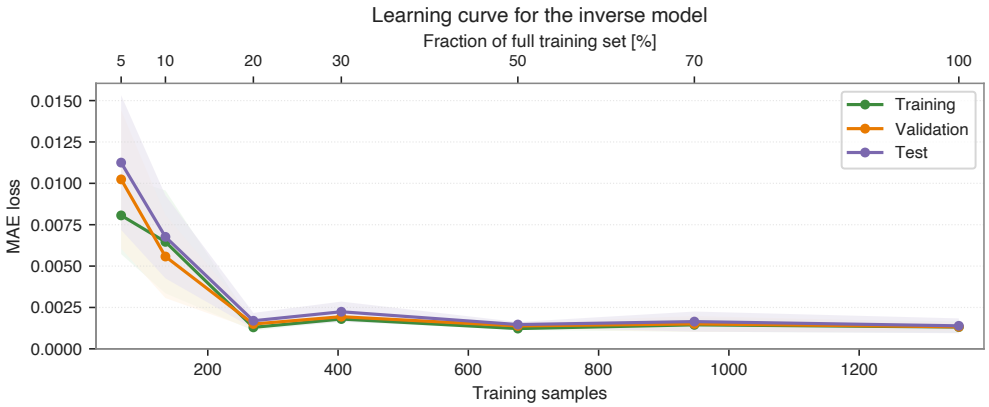


Fig. 11: Learning curve analysis for the transmon cross inverse model. Validation (orange), test (purple), and training (green) MAE losses are plotted against training set size, expressed both as the number of samples (bottom axis) and as a fraction of the full training set (top axis). Shaded bands indicate $\pm 1\sigma$ over multiple random seeds. Loss drops quickly up to about 20% of the data, or roughly 250 samples, and then plateaus. This suggests that the model is close to saturated on the current SQuADDS sampling pattern, so further improvement is more likely to come from targeted sampling of sparse or high-error regions than from simply adding more randomly drawn samples.

For the sample in Fig. 12, the target qubit frequency is 4.20 GHz and the target anharmonicity is -157.45 MHz. The predicted geometry lands at 4.20 GHz and -157.33 MHz, giving a 0.04% error in ω_q and a 0.08% error in α, even though the cross length moves from 250.0 µm to 259.0 µm and the claw length moves from 100.0 µm to 148.2 µm. The predicted layout should therefore be read as an alternative valid design, not as an attempt to reproduce the SQuADDS reference geometry.

6 Discussion

Among the three components studied here, the transmon cross is the most straightforward inverse problem. It has a small number of continuous outputs, a dense SQuADDS dataset to learn from, and a reasonably direct physical mapping from geometry to E_C and therefore to the qubit Hamiltonian. The NCap and the resonator are more challenging because their datasets are smaller.

The results also make the non-uniqueness of the inverse problem hard to ignore. For many held-out targets, the network predicts a geometry that looks nothing like the SQuADDS reference but still reconstructs the target Hamiltonian to within the few-percent simulation-to-measurement error floor. That behavior is appropriate for a one-to-many inverse problem. The model should be judged by the achieved Hamiltonian, not by whether it reproduces one database geometry. It also means that evaluating inverse models for qubit design on pointwise geometry metrics, such as distance to the reference design, would be misleading. The achieved Hamiltonian is the appropriate grading metric, and the same point has been made repeatedly in the broader inverse-design literature [8, 9].

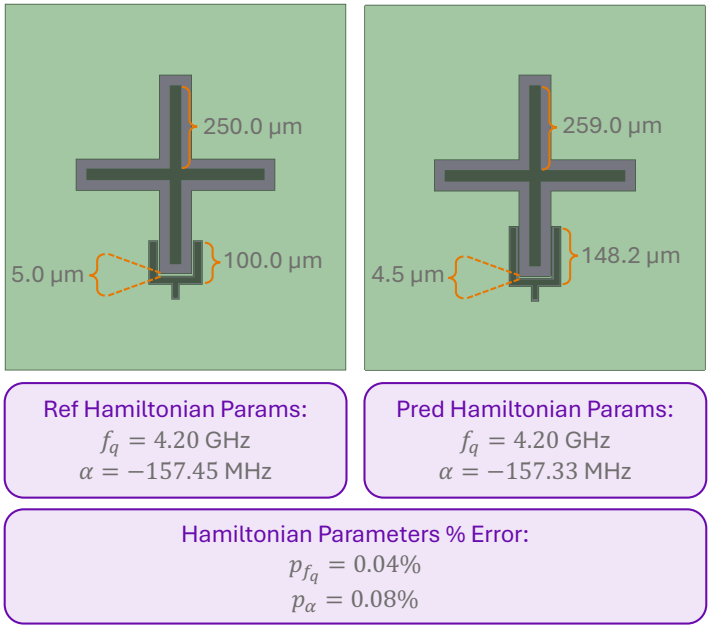


Fig. 12: Reference and model-predicted transmon-cross designs for a representative sample. Although the predicted geometry differs noticeably from the SQuADDS reference, the reconstructed Hamiltonian parameters closely match the target, illustrating the one-to-many nature of the inverse mapping.

In its current form, the pipeline is most useful as a proposal tool rather than as a complete replacement for simulation. The network proposes a candidate layout, one forward simulation validates it, and small manual refinements can follow if needed. This shifts the expensive part of iteration from repeated EM solves to fast network evaluation, while still leaving room for final simulation checks and manual refinement.

A natural next step is to replace the direct-regression inverse model with a generative model that can represent a distribution of valid geometries rather than one point estimate, using active learning to enrich SQuADDS in the sparse regions flagged by the stress test, and adding soft penalty terms that push the network away from regions of parameter space that look fine on paper but are unreliable in practice.

7 Limitations and future work

The main limitation is not simply the total number of training samples, but how those samples cover the design space. SQuADDS is dense in some regions and sparse elsewhere, and this uneven coverage appears in both the nearest-neighbor stress test and the learning curve plateau. Adding more samples in the same pattern would probably have limited benefit. A better route is targeted data generation in regions where the surrogate is least supported or least reliable, although each new Ansys simulation is still expensive.

Other limitations include uncertainty from simulation settings and meshing, limited parameter ranges and fabrication constraints, and a domain gap between simulated and measured devices. The fabrication-to-simulation gap matters especially for α, which depends on E_C through the full qubit capacitance, so that even small lithographic errors can shift E_C by a few percent.

8 Conclusion

This work demonstrates Hamiltonian-targeted inverse prediction of Quantum Metal parameters for a transmon cross, using an inverse MLP paired with a learned Ansys forward surrogate. Closed-loop validation measures whether the predicted layouts recover the target quantities, with mean percent errors of about 1% in ω_q and 2% in α on the held-out test set. Those errors are close to the simulation-to-measurement error floor reported for SQuADDS [4], suggesting that additional gains on this benchmark will likely come from better targeted coverage of sparse regions in the design space rather than from simply increasing the size of the training set. At the same time, the fully neural pipeline runs in about 56 ms for a single query on CPU, and in batch mode reconstructs Hamiltonians at about 0.235 ms per sample on the same CPU class used for the Ansys benchmark, while a single Ansys Q3D capacitance extraction still requires roughly ∼2 minutes per sample (Section 4.6). The more relevant comparison is the full iterative design loop, where reaching a target Hamiltonian usually requires many simulation cycles and substantial designer time. Comparing to the kind of iterative design sweep that is routine in SQuADDS-style workflows, the learned pipeline is therefore several orders of magnitude faster per design query, and the one-time cost of generating the training data breaks even after a few hundred queries. Taken together, the results support component-level inverse design as a useful starting point in a small-data regime. Future work will focus on inverse non-uniqueness and on improving reliability with generative models and active learning, will pursue experimental validation on fabricated devices to close the loop from target Hamiltonian to measured device, and will revisit the NCap (Appendix D) and resonator (Appendix E) components with expanded datasets.

Appendix A Testing pipeline (Ansys in the loop)

Figure A1 shows the Ansys-in-the-loop validation path. In this mode, the predicted design is passed through the standard Python-to-Ansys workflow rather than through the learned forward surrogate.

Appendix B Inverse only TransmonCross model (without Ansys surrogate)

For completeness, this appendix reports how the inverse TransmonCross model performs when evaluated in closed loop with Ansys Q3D and scQubits directly, without the forward surrogate. This is the configuration that matches the testing pipeline in Fig. A1. The headline surrogate result in the main text (Section 5.2, Fig. 7) is the

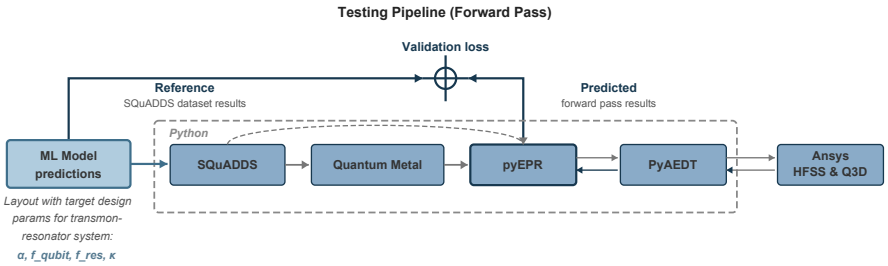


Fig. A1: Testing pipeline used for forward validation of the inverse model, with Ansys HFSS and Q3D retained in the loop.

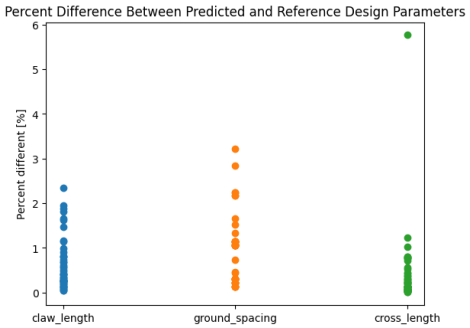


Fig. B2: Inverse only transmon cross performance without the Ansys surrogate.

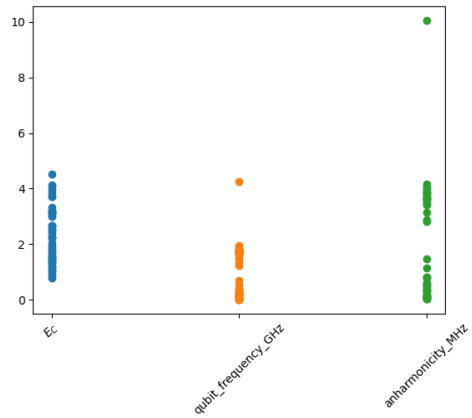


Fig. B3: Additional performance result for the inverse only transmon cross model.

stronger demonstration, so we retain the inverse-only baseline here as a reference point.

Appendix C Transmon-cross capacitance comparison

The main text reports the inverse plus surrogate transmon cross results at the Hamiltonian level. This appendix reports the matching capacitance-level comparison on the same samples, showing extracted capacitance matrix elements for predicted versus reference designs.

Appendix D NCap results

This appendix reports the corresponding results for the NCap coupling capacitor. The NCap dataset has only 430 total samples, split 301 / 64 / 65 for training, validation, and testing (Table 2), which is about a factor of 4.5 smaller than the transmon cross dataset. Because of that smaller dataset, we treat the NCap results as supporting results rather than as the main demonstration.

Unlike the transmon cross, which in the main text is trained with a

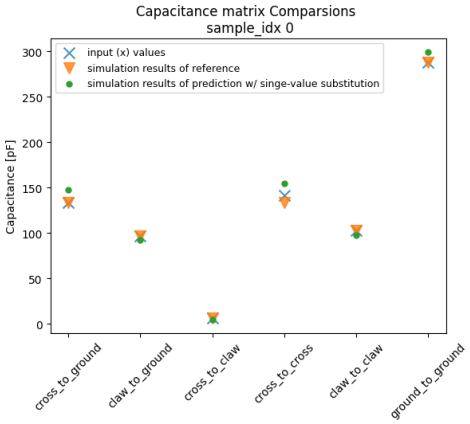


Fig. C4: Capacitance-level comparison of predicted and reference transmon cross designs (same samples as Fig. 7).

Hamiltonian-level surrogate loss, the NCap inverse model is trained with a *capacitance*-level surrogate loss. The inputs are the six independent entries of the capacitance matrix, and the inverse model is chained with a frozen forward surrogate that maps Quantum Metal parameters back to capacitance. This choice reflects the fact that the NCap coupler's role in the circuit is captured directly by its capacitance matrix, with no intermediate Hamiltonian abstraction needed. The six capacitance inputs are `top_to_top`, `top_to_bottom`, `top_to_ground`, `bottom_to_bottom`, `bottom_to_ground`, and `ground_to_ground`.

D.1 Categorical encoding and exists parameters

Not every Quantum Metal parameter is continuous. The NCap finger count, for example, is an integer that only takes values in a small discrete set. We compared one hot and linear encodings for these categorical outputs, and one hot gave cleaner predictions, mostly because the network did not have to invent an ordering that did not exist in the layout. For the transmon cross, every predicted parameter is continuous, so the categorical encoding didn't come into play. The only categorical output we had to handle was the NCap finger count in Appendix D.

Parameters that do not exist for some designs get handled separately. Some layouts in the wider SQuADDS database have an optional feature that only shows up when another setting is active. For those we add a binary exists mask alongside the parameter, and at postprocessing time we drop the parameter whenever its mask is off. That way the network does not get penalized for guessing values for fields that were going to be thrown away anyway.

D.2 Outputs and Quantum Metal parameters predicted by the NCap model

D.3 Hyperparameter tuning

The NCap hyperparameter search follows the same Keras Tuner protocol as the transmon cross, but over a wider range of widths and depths to handle the higher dimensional output. The best validation loss was 0.1254, larger than for the transmon cross, as expected from the smaller dataset and the mixture of continuous and categorical outputs.

D.4 Inverse + surrogate performance

Appendix E Resonator results

This appendix reports the corresponding results for the cavity or resonator

Table D1: Capacitance matrix inputs (**x**) and Quantum Metal outputs (**y**) for the NCap inverse model. The model outputs 13 values, three continuous design parameters and a one hot encoding of `finger_count` over the 10 supported values. Ranges for continuous quantities are the min and max observed in the full SQuADDS-derived dataset (train + val + test).

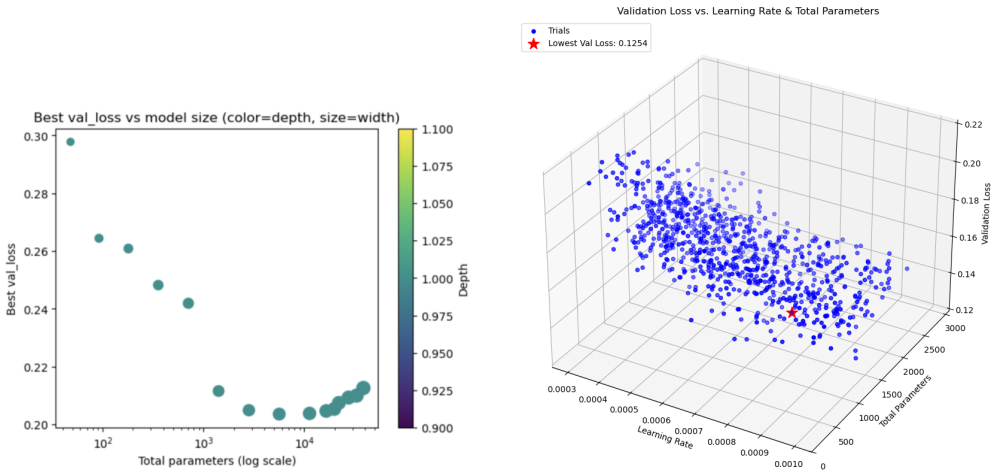
Parameter	Value range	Units
Inputs, capacitance matrix entries (Ansys Q3D)		
`top_to_top`	14.19 to 95.19	fF
`top_to_bottom`	0.36 to 54.48	fF
`top_to_ground`	12.80 to 41.68	fF
`bottom_to_bottom`	12.70 to 145.6	fF
`bottom_to_ground`	12.10 to 90.45	fF
`ground_to_ground`	56.76 to 170.2	fF
Outputs, continuous Quantum Metal parameters		
`design_options.cap_gap`	2.1 to 4.1	µm
`design_options.cap_width`	4.9 to 14.9	µm
`design_options.finger_length`	15.9 to 45.9	µm
Outputs, one hot `finger_count` (categorical)		
`design_options.finger_count_1`	0/1	cat.
`design_options.finger_count_2`	0/1	cat.
`design_options.finger_count_3`	0/1	cat.
`design_options.finger_count_4`	0/1	cat.
`design_options.finger_count_5`	0/1	cat.
`design_options.finger_count_6`	0/1	cat.
`design_options.finger_count_7`	0/1	cat.
`design_options.finger_count_8`	0/1	cat.
`design_options.finger_count_9`	0/1	cat.
`design_options.finger_count_10`	0/1	cat.

(`claw_RouteMeander`) component, which uses an Ansys HFSS eigenmode solver rather than Q3D for forward extraction. The resonator dataset has 810 total samples split 567 / 121 / 122 for training, validation, and testing (Table 2), about half the size of the transmon-cross dataset. We therefore report it as a supporting result in the appendix.

E.1 Outputs and Quantum Metal parameters predicted by the resonator model

E.2 Hyperparameter tuning

The best validation loss from the resonator Keras Tuner search was 0.0871, between the transmon-cross and NCap values and consistent with the intermediate dataset size.



(a) Parameter sweep.

(b) Keras Tuner search metrics.

Fig. D5: Hyperparameter tuning diagnostics for the NCap inverse model.

Table E2: Quantum Metal output parameters (**y**) predicted by the resonator inverse model.

Quantum Metal Parameter	Value range	Units
Cavity model, claw_RouteMeander (eigenmode)		
`design_options.claw_opts.connection_pads.readout .claw_length`	70 to 578	μm
`design_options.claw_opts.connection_pads.readout .ground_spacing`	4 to 10	μm
`design_options.cpw_opts.total_length`	2700 to 4700	μm
`design_options.cpw_opts.meander.asymmetry`	-250 to 133	μm
`design_options.cplr_opts.coupling_length`	100 to 500	μm

E.3 Forward surrogate performance

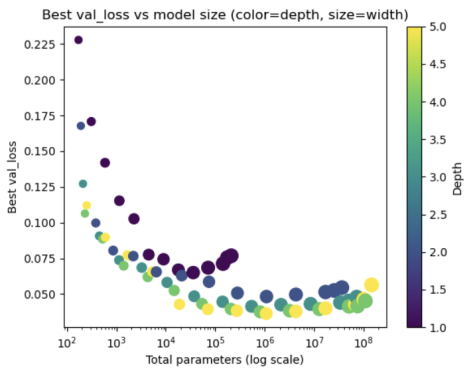
E.4 Inverse + surrogate performance

Appendix F Resonator eigenmode setup

We do not include a separate Q3D setup table because the convergence error is the relevant quantity for the capacitance runs, and that filtering step is described in Section 3.2.

Supplementary information. No supplementary information is included with this version.

Acknowledgements. The authors would like to thank Jonathan Asaadi and Daniel Baxter for helpful discussions and encouragement in the preparation of this manuscript. The authors would also



(a) Parameter sweep.

plots/hypertuner_search_metrics/3D_Val_Loss_vs_Learni

(b) Keras Tuner search metrics.

Fig. E6: Hyperparameter tuning diagnostics for the resonator inverse model.

Table F3: Ansys HFSS eigenmode setup fields to be filled from the final resonator sweep log before submission.

Parameter	**Value**	**Rationale**
Max passes	TBD	Convergence criterion on mode frequency.
Min converged passes	TBD	Stability of the adaptive mesh refinement.
Δf tolerance	TBD	Target relative change between passes.
Basis order	TBD	Accuracy vs. runtime trade off.
Mesh seeding	TBD	Used on critical geometry (coupling region, CPW).
Port and boundary setup	TBD	As used in SQuADDS reference runs.

like to thank Samuel Stein, Ang Li, and Chenxu Liu at Pacific Northwest National Laboratory for early discussions regarding this work. Olivia Seidel would also like to thank Giuseppe Di Guglielmo for valuable mentoring regarding the construction and use of an MLP for a previous project, which was later applied to this work. The authors used Anthropic's Claude (Opus 4.7) only for language editing and clarity checks. All technical content, analysis, and conclusions are the authors' own. This manuscript has been authored by Fermi Research Alliance, LLC under Contract No. DE-AC02-07CH11359 with the U.S. Department of Energy, Office of Science, Office of High Energy Physics.

Declarations

- **Funding** Not applicable.
- **Conflict of interest / Competing interests** The authors declare that they have no competing interests.
- **Ethics approval and consent to participate** Not applicable.

- **Consent for publication** Not applicable.
- **Data availability** The datasets used in this study are derived from SQuADDS. Processed splits and model artifacts will be made available with the public code release.
- **Materials availability** Not applicable.
- **Code availability** The code used to produce the results in this paper is available at https://github.com/CosmiQuantum/ML_qubit_design.
- **Author contribution** Author contributions will be finalized before submission.

References

[1] Qiskit Metal. https://qiskit.org/metal

[2] Minev, Z.K., Leghtas, Z., Mundhada, S.O., Christakis, L., Pop, I.M., Devoret, M.H.: Energy-participation quantization of josephson circuits. npj Quantum Information **7**(1), 131 (2021) https://doi.org/10.1038/s41534-021-00461-8

[3] Groszkowski, P., Koch, J.: Scqubits: a python package for superconducting qubits. Quantum **5**, 583 (2021)

[4] Shanto, S., Kuo, A., Miyamoto, C., Zhang, H., Maurya, V., Vlachos, E., Hecht, M., Shum, C.W., Levenson-Falk, E.M.: SQuADDS: a validated design database and simulation workflow for superconducting qubit design. Quantum **8**, 1465 (2024) https://doi.org/10.22331/q-2024-09-09-1465

[5] Levenson-Falk, E.M., Shanto, S.A.: A review of design concerns in superconducting quantum circuits. arXiv:2411.16967 (2025)

[6] Koch, J., Yu, T.M., Gambetta, J., Houck, A.A., Schuster, D.I., Majer, J., Blais, A., Devoret, M.H., Girvin, S.M., Schoelkopf, R.J.: Charge-insensitive qubit design derived from the Cooper pair box. Phys. Rev. A **76**(4), 042319 (2007) https://doi.org/10.1103/PhysRevA.76.042319

[7] Peurifoy, J., Shen, Y., Jing, L., Yang, Y., Cano-Renteria, F., DeLacy, B.G., Joannopoulos, J.D., Tegmark, M., Soljacic, M.: Nanophotonic particle simulation and inverse design using artificial neural networks. Sci. Adv. **4**(6), 4206 (2018) https://doi.org/10.1126/sciadv.aar4206

[8] Liu, D., Tan, Y., Khoram, E., Yu, Z.: Training deep neural networks for the inverse design of nanophotonic structures. ACS Photonics **5**(4), 1365–1369 (2018) https://doi.org/10.1021/acsphotonics.7b01377

[9] Molesky, S., Lin, Z., Piggott, A.Y., Jin, W., Vuckovic, J., Rodriguez, A.W.: Inverse design in nanophotonics. Nat. Photonics **12**(11), 659–670 (2018) https://doi.org/10.1038/s41566-018-0246-9

[10] Wong, H.Y., Dhillon, P., Beck, K.M., Rosen, Y.J.: A simulation methodology for superconducting qubit readout fidelity. Solid-State Electronics **201**, 108582 (2023) https://doi.org/10.1016/j.sse.2022.108582

[11] Lu, A., Wong, H.Y.: Rapid simulation framework for superconducting qubit readout system inverse design and optimization. In: 2024 International Conference on Simulation of Semiconductor Processes

and Devices (SISPAD), pp. 1–4 (2024). https://doi.org/10.1109/SISPAD62626.2024.10733335